CLASE 1

Introducción y fundamentos de archivos

Lectura de archivos y procesamiento de información en Java



Temas a tratar:

- ¿Qué son los archivos y para qué sirven?
- Tipos de archivos
- Diferencia entre RAM y almacenamiento permanente
- Lectura de archivos: read() y readLine()

Grupo 5

Sandra Alulema | Anthony Armas | Joselyn Guaman | Valentina Ortiz | Andrés Vayas

Ambato, Ecuador · Noviembre de 2026

CLASE 1

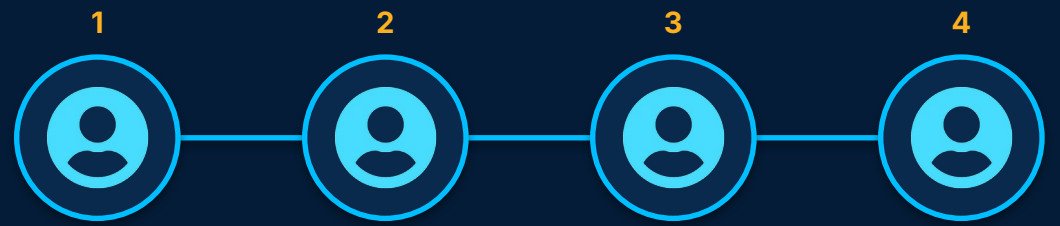
Ruta de la clase 1

Introducción y fundamentos de archivos · 4 participaciones

Información clave

1. Vale: qué son los archivos, para qué sirven y tipos de archivos (texto, imágenes, documentos y binarios).
2. Jos: diferencia entre RAM y almacenamiento permanente, y cómo funciona la lectura de archivos.
3. Sandra: leer, escribir y almacenar información, más los métodos read() y readLine().
4. Anthony: paquete java.io, rutas absolutas y relativas, y clases FileReader y BufferedReader.

ORDEN DE PARTICIPACIÓN



CLASE 1 · Archivos en Java

OBJETIVO

Entender cómo un programa Java guarda y recupera datos usando archivos.

¿Qué son los archivos?

Concepto y función dentro de un sistema informático

Información clave

1. Un archivo es un conjunto de datos relacionados que se almacena en un dispositivo.
2. Tiene un nombre y una ubicación determinados (ej.: notas.txt en la carpeta datos).
3. Conserva la información de forma duradera, incluso después de cerrar el programa.
4. En programación, permite que una aplicación guarde y reutilice datos.



`notas.txt`

conjunto de datos relacionados

- **Nombre identificable**
- **Ubicación específica**
- **Almacenamiento duradero**

IDEA CLAVE

El archivo le da memoria a un programa: los datos sobreviven aunque se cierre.

¿Para qué sirven los archivos?

Ejemplos de uso en la vida diaria y en los programas

Información clave

1. Guardar documentos, imágenes y configuraciones.
2. Mantener registros de ventas, notas o inventarios.
3. Compartir información entre usuarios y aplicaciones.
4. Conservar datos para consultarlos en otro momento.



El programa se cierra, el archivo permanece

EJEMPLO PRÁCTICO

Un sistema académico guarda las notas en un archivo; al volver a ejecutarse, las lee y recupera los datos.

Tipos de archivos

Se clasifican según la información que contienen

Información clave

1. Texto: caracteres legibles. Ej.: .txt, .csv, .java.
2. Imágenes: datos gráficos. Ej.: .png, .jpg, .gif.
3. Documentos: texto con formato y otros elementos. Ej.: .pdf, .docx, .xlsx.
4. Binarios: datos en la representación de la máquina; no se leen como texto. Ej.: .exe, .class, .jar.

LEGIBLES

`.txt` `.csv` `.java` `.png` `.jpg` `.gif` `.pdf` `.docx` `.xlsx`

BINARIOS

`.exe` `.class` `.jar` No se leen como texto

DATO CURIOSO

Un archivo .java es de texto; al compilarlo se genera un .class, que es binario.

RAM vs. almacenamiento permanente

Dos espacios con funciones diferentes

Información clave

1. RAM: rápida; permite trabajar con variables y objetos mientras el programa se ejecuta.
2. La RAM es volátil: pierde su contenido al cerrar el programa o apagar el equipo.
3. Almacenamiento permanente: disco duro, SSD y memoria USB.
4. Conserva los datos aunque se apague el equipo; es más lento que la RAM.

RAM

espacio de trabajo temporal

Velocidad

muy rápida

Permanencia

volátil: se borra al cerrar

Disco · SSD · USB

guardado a largo plazo

Velocidad

más lenta

Permanencia

permanente: conserva los datos

ANALOGÍA

La RAM es el escritorio donde trabajas; el disco es el archivador donde guardas.

¿Cómo funciona la lectura de archivos?

Del archivo guardado en el disco al programa, paso a paso

Información clave

1. El programa abre el archivo indicando su ruta.
2. Un lector (FileReader) toma los caracteres del archivo, guardado en el almacenamiento permanente.
3. Los datos pasan a la RAM, donde el programa los procesa.
4. Al terminar, se cierra el archivo con close() para liberar el recurso.



Se lee en orden, de principio a fin

IDEA CLAVE

Leer es traer datos del almacenamiento permanente a la RAM para trabajar con ellos.

CLASE 1

SANDRA

Leer, escribir y almacenar

Tres operaciones básicas al trabajar con archivos

Información clave

1. Leer: traer datos guardados en un archivo hacia el programa. Ej.: consultar notas.txt.
2. Escribir: enviar información del programa hacia un archivo. Ej.: guardar el nombre de un estudiante.
3. Almacenar: dejar los datos guardados de forma permanente en un dispositivo.
4. Son operaciones distintas que casi siempre se usan juntas.



LEER

archivo → programa

Ej.: consultar notas.txt



ESCRIBIR

programa → archivo

Ej.: guardar un nombre



ALMACENAR

datos guardados

Ej.: registro para después

RECUERDA

Leer: archivo → programa · Escribir: programa → archivo.

CLASE 1

SANDRA

Métodos read() y readLine()

Dos formas de leer el contenido de un archivo de texto

Información clave

1. read(): lee un solo carácter y lo devuelve como int (su código).
2. read() devuelve -1 cuando ya no quedan caracteres por leer.
3. readLine(): lee una línea completa y la devuelve como String (método de BufferedReader).
4. readLine() devuelve null al llegar al final del archivo.
5. Ambos pueden lanzar IOException, por lo que se deben manejar.

`read()`

LEE
1 carácter

DEVUELVE
`int (su código)`

AL FINAL
`-1`

`readLine()`

LEE
1 línea completa

DEVUELVE
`String`

AL FINAL
`null`

Ambos pueden lanzar IOException

EJEMPLO

```
String linea = br.readLine();
```

CLASE 1

ANTHONY

Archivos en Java: paquete java.io

Herramientas para realizar operaciones de entrada y salida

Información clave

1. Es el paquete de Java que reúne clases para operaciones de entrada y salida.
2. Permite leer información desde archivos o escribirla en ellos.
3. Se puede importar completo o solo las clases necesarias.
4. Aporta clases para acceder a archivos y gestionar lectura y escritura.

```
import java.io.*;
```

CLASES QUE ENCONTRARÁS EN java.io

`File` `FileReader` `FileWriter` `BufferedReader` `IOException`

SINTAXIS

```
import java.io.*;   o bien   import java.io.File;
```

ANTHONY

Rutas absolutas y relativas

Formas de indicar dónde se encuentra un archivo

Información clave

1. Absoluta: ubicación completa desde la raíz. Ej.: C:/Users/estudiante/datos/notas.txt.
2. Es específica del equipo y de su estructura de carpetas.
3. Relativa: parte de la carpeta del proyecto. Ej.: datos/notas.txt.
4. Más portable: funciona al mover el proyecto si se conserva su estructura.

ABSOLUTA

Depende del equipo

`C:/` › `Users` › `estudiante` › `datos` › `notas.txt`

RELATIVA

Portable ✓

`proyecto` › `datos` › `notas.txt`

RECOMENDACIÓN

Usa rutas relativas para que tu proyecto funcione en otro equipo.

CLASE 1

ANTHONY

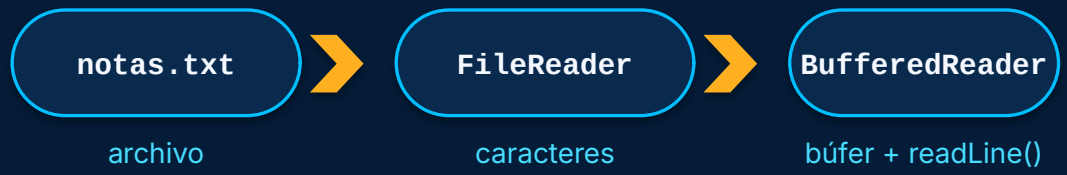
FileReader y BufferedReader

Clases de java.io para leer archivos de texto

Información clave

1. FileReader: lee caracteres de un archivo de texto a partir de su ruta.
2. Si el archivo no existe, lanza FileNotFoundException.
3. BufferedReader: envuelve al FileReader y lee con un búfer, de forma más eficiente.
4. Añade readLine() para leer línea por línea; se cierra con close().

```
try (BufferedReader br = new BufferedReader(
   new FileReader("datos/notas.txt")) {
   String linea = br.readLine();
} catch (IOException e) { }
```



SINTAXIS

```
BufferedReader br = new BufferedReader(new FileReader(ruta));
```

Cierre de la clase 1

Lo esencial que debemos recordar

Información clave

1. Los archivos conservan datos más allá de la ejecución; hay de texto, imágenes, documentos y binarios.
2. La RAM es volátil; el almacenamiento permanente conserva la información.
3. Leer, escribir y almacenar son operaciones distintas; read() lee un carácter y readLine() una línea.
4. java.io reúne las herramientas; FileReader y BufferedReader leen archivos de texto con rutas absolutas o relativas.

- Los archivos persisten
- Texto · imagen · doc · binario
- RAM volátil · disco permanente
- Leer · escribir · almacenar
- read() y readLine()
- Paquete java.io
- Rutas absolutas y relativas
- FileReader y BufferedReader

PARA RECORDAR

read() lee un carácter; readLine() lee una línea completa con BufferedReader.